

Collections and Containers

Simon Frost

Overview

RDF provides two mechanisms for grouping resources: **Collections** (ordered, closed lists using `rdf:first/rdf:rest`) and **Containers** (open groups using `rdf:Bag`, `rdf:Seq`, `rdf:Alt`). This vignette covers both.

```
using RDFLib
ex = Namespace("http://example.org/")
```

```
Namespace("http://example.org/")
```

RDF Collections (Lists)

An RDF Collection is a linked list using `rdf:first` and `rdf:rest` properties, terminated by `rdf:nil`.

Creating Collections

```
g = RDFGraph()
bind!(g, "ex", ex)

# Add a collection (list) of items linked to a subject
items = Identifier[Literal("Mercury"), Literal("Venus"), Literal("Earth"), Literal("Mars")]
add_collection!(g, ex("innerPlanets"), ex("members"), items)

println(serialize(g, TurtleFormat()))
```

```

@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:innerPlanets ex:members [ rdf:first "Mercury" ;
                             rdf:rest [ rdf:first "Venus" ;
                                         rdf:rest [ rdf:first "Earth" ;
                                                     rdf:rest [ rdf:first "Mars" ;
                                                                 rdf:rest rdf:nil ] ] ] ] .

```

Extracting Collections

```

# Find the list head and retrieve items
list_head = first(objects(g, ex("innerPlanets"), ex("members")))
extracted = collect_list(g, list_head)
println("Inner planets:")
for item in extracted
    println("  ", item)
end

```

```

Inner planets:
  Literal("Mercury")
  Literal("Venus")
  Literal("Earth")
  Literal("Mars")

```

Turtle Shorthand

In Turtle, collections are written with parentheses:

```

g2 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:recipe ex:ingredients ( "flour" "sugar" "eggs" "butter" ) .
""", TurtleFormat())

# Find the list head and extract items
for t in triples(g2, (ex("recipe"), ex("ingredients"), nothing))

```

```

    items = collect_list(g2, t.object)
    println("Ingredients: ", items)
end

```

Ingredients: Identifier[Literal("flour"), Literal("sugar"), Literal("eggs"), Literal("butter")]

Collection Views

CollectionView provides indexed access to RDF lists:

```

cv = collection_view(g2, collect(triples(g2, (ex("recipe"), ex("ingredients"), nothing))))[1]
println("First ingredient: ", cv[1])
println("Last ingredient:  ", cv[length(cv)])
println("Number of items:  ", length(cv))

```

First ingredient: Literal("flour")
 Last ingredient: Literal("butter")
 Number of items: 4

RDF Containers

Containers are open-ended groups using membership properties (`rdf:_1`, `rdf:_2`, etc.).

Bag (Unordered)

```

g3 = RDFGraph()
bind!(g3, "ex", ex)

# Create a Bag of authors
authors = Identifier[Literal("Alice"), Literal("Bob"), Literal("Carol")]
add_container!(g3, ex("paper1Authors"), :Bag, authors)

println(serialize(g3, TurtleFormat()))

```

```

@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

```

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
ex:paper1Authors a rdf:Bag ;
    rdf:_1 "Alice" ;
    rdf:_2 "Bob" ;
    rdf:_3 "Carol" .
```

Seq (Ordered)

```
g4 = RDFGraph()
bind!(g4, "ex", ex)

# Create an ordered sequence
steps = Identifier[Literal("Preheat oven"), Literal("Mix ingredients"), Literal("Bake for 30 min")]
add_container!(g4, ex("recipe_steps"), :Seq, steps)

println(serialize(g4, TurtleFormat()))
```

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
ex:recipe_steps a rdf:Seq ;
    rdf:_1 "Preheat oven" ;
    rdf:_2 "Mix ingredients" ;
    rdf:_3 "Bake for 30 min" .
```

Extracting Container Items

```
items = collect_container(g4, ex("recipe_steps"))
println("Recipe steps:")
for (i, item) in enumerate(items)
    println("  $i. $item")
end
```

Recipe steps:

1. Literal("Preheat oven")
2. Literal("Mix ingredients")
3. Literal("Bake for 30 min")

Alt (Alternatives)

```
g5 = RDFGraph()
bind!(g5, "ex", ex)

# Alternatives (e.g., translations)
translations = Identifier[
    Literal("Hello"; lang="en"),
    Literal("Bonjour"; lang="fr"),
    Literal("Hola"; lang="es")
]
add_container!(g5, ex("greeting"), :Alt, translations)

println(serialize(g5, TurtleFormat()))
```

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
ex:greeting a rdf:Alt ;
    rdf:_1 "Hello"@en ;
    rdf:_2 "Bonjour"@fr ;
    rdf:_3 "Hola"@es .
```

Collections in SPARQL

```
g6 = parse_rdf("""
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

```

    ex:favorites ex:list ( "alpha" "beta" "gamma" "delta" ) .
""" , TurtleFormat())

# Query list items using property paths
results = sparql_query(g6, """
    PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
    PREFIX ex: <http://example.org/>

    SELECT ?item WHERE {
        ex:favorites ex:list/rdf:rest*/rdf:first ?item .
    }
    """ )

println("Favorites:")
for row in results
    println("  ", row["item"])
end

```

```

Favorites:
  Literal("alpha")
  Literal("beta")
  Literal("gamma")
  Literal("delta")

```

Nested Collections

Collections can contain other collections or complex structures:

```

g7 = parse_rdf("""
    @prefix ex: <http://example.org/> .

    ex:matrix ex:rows (
        ( 1 2 3 )
        ( 4 5 6 )
        ( 7 8 9 )
    ) .
    """ , TurtleFormat())

println("Matrix graph: $(length(g7)) triples")
println(serialize(g7, TurtleFormat()))

```

```

Matrix graph: 25 triples
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:matrix ex:rows [ rdf:first [ rdf:first 1 ;
    rdf:rest [ rdf:first 2 ;
    rdf:rest [ rdf:first 3 ;
    rdf:rest rdf:nil ] ] ] ;
    rdf:rest [ rdf:first [ rdf:first 4 ;
    rdf:rest [ rdf:first 5 ;
    rdf:rest [ rdf:first 6 ;
    rdf:rest rdf:nil ] ] ] ;
    rdf:rest [ rdf:first [ rdf:first 7 ;
    rdf:rest [ rdf:first 8 ;
    rdf:rest [ rdf:first 9 ;
    rdf:rest rdf:nil ] ] ] ;
    rdf:rest rdf:nil ] ] ] .

```